

The `Shape` class inherits from the `ABC` class ❶, telling Python to prevent client code from instantiating a `Shape` object directly. Any attempt to do so results in the following error message:

```
TypeError: Can't instantiate abstract class Shape with abstract methods
clickedInside, draw, getArea
```

The `__init__()` ❷ and `getType()` ❸ methods contain code that will be shared by all subclasses of `Shape`.

The `clickedInside()` ❹, `getArea()` ❺, and `draw()` ❻ methods are all preceded by the `@abstractmethod` decorator. This decorator indicates that these methods *must* be overwritten by all subclasses of `Shape`. Since these methods in this abstract class will never run, the implementation here consists only of `raise NotImplementedError` to further emphasize that the method doesn't do anything.

Let's extend the shape demonstration program to add a new `Rectangle` class, as shown in [Listing 10-11](#). The `Rectangle` class inherits from the abstract `Shape` class and therefore must implement the `clickedInside()`, `getArea()`, and `draw()` methods. I'll make an intentional error in this subclass to show what happens.

File: InheritedShapes/Rectangle.py

```
# Rectangle class

import pygame
from Shape import *

class Rectangle(Shape):

    def __init__(self, window, maxWidth, maxHeight):
        super().__init__(window, 'Rectangle', maxWidth, maxHeight)
        self.width = random.randrange(10, 100)
        self.height = random.randrange(10, 100)
        self.rect = pygame.Rect(self.x, self.y, self.width, self.height)

    def clickedInside(self, mousePoint):
        clicked = self.rect.collidepoint(mousePoint)
        return clicked
```